A+ LAB SERIES V5

Lab 20: Writing Basic Scripts

Document Version: 2026-01-02

Material in this Lab Aligns to the Following	
CompTIA A+ (220-1201) Exam Objectives	
CompTIA A+ (220-1202) Exam Objectives	4.8: Explain the basics of scripting.

Contents

Introduction

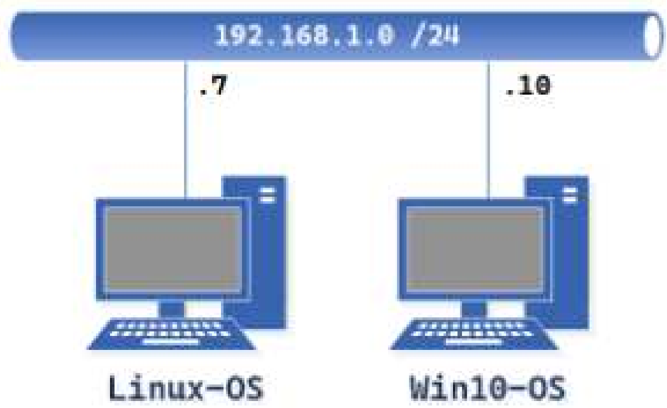
Scripts are primarily used to automate administration functions. This saves time and gives administrators a way to control how tasks are done, reducing the risk of errors and making tasks more time-efficient. In this lab, you will write scripts in different scripting languages to perform some simple tasks.

Objective

In this lab, you will write basic scripts in different scripting languages to understand how each language handles automating tasks.

- Create a Windows Batch File (Script) using Notepad.
- Test the Windows Batch File using Command Prompt.
- Create a PowerShell script and test the PowerShell script.
- Create a BASH script using Linux Mint and test the BASH script.

Lab Topology



Lab Settings

The information in the table below will be needed to complete the lab. The task sections below provide details on the use of this information.

Virtual Machine	**IP Address**	**Account** (if needed)	**Password** (if needed)
Linux-OS	192.168.1.7 / 24	`sysadmin`	`NDGlabpass123!`
Win10-OS	192.168.1.10 / 24	`sysadmin`	`NDGlabpass123!`

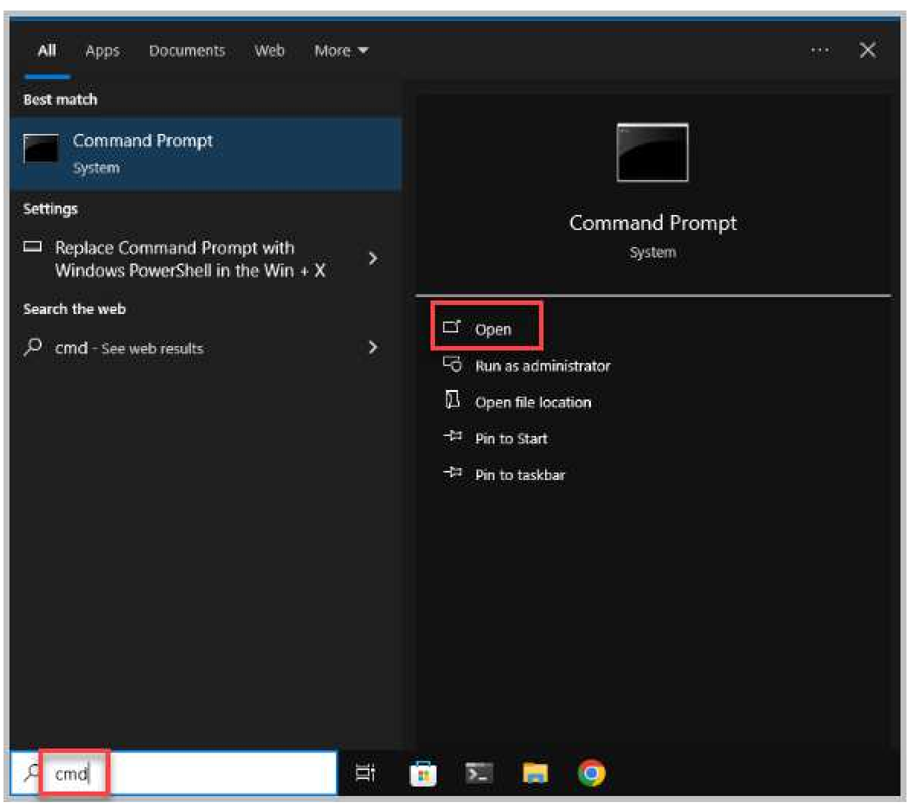
1 Creating and Testing a Windows Batch File (Script)

A batch file is a special kind of text file with a .bat extension that contains one or more commands a Windows command prompt understands and executes in sequence to perform tasks. Entering commands manually, one at a time, will achieve the same results, but a batch file simplifies retyping commands, saving time and reducing the risk of typos. Batch files are written in text editors. In this exercise, you will use Notepad to create the script file.

1. Open a console connection to the **Win10-OS** system.

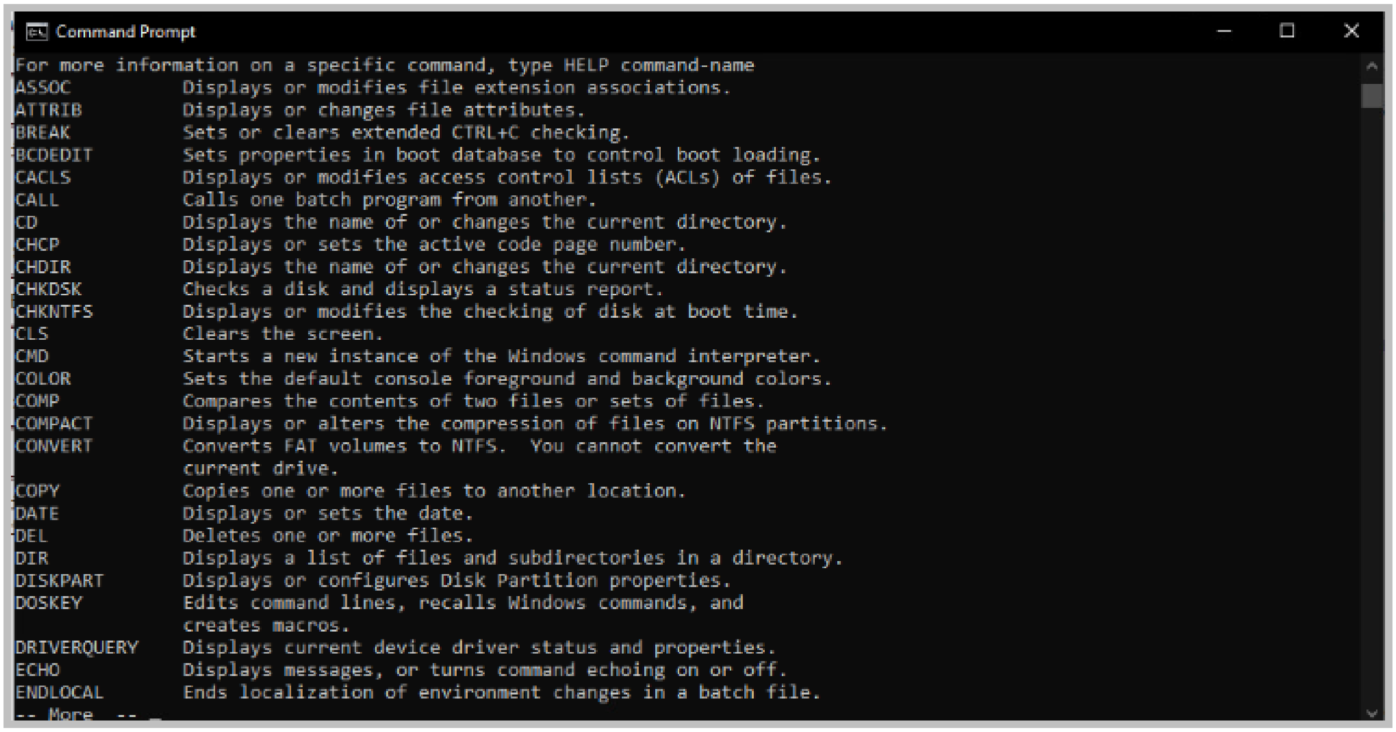
> To launch the console for a lab device, you can click on the respective tab in the navigation bar at the top or navigate to the *Topology* section and click the lab device's corresponding image.

2. In the **search** textbox located in the taskbar, type `cmd` in the search field. Select **Open**



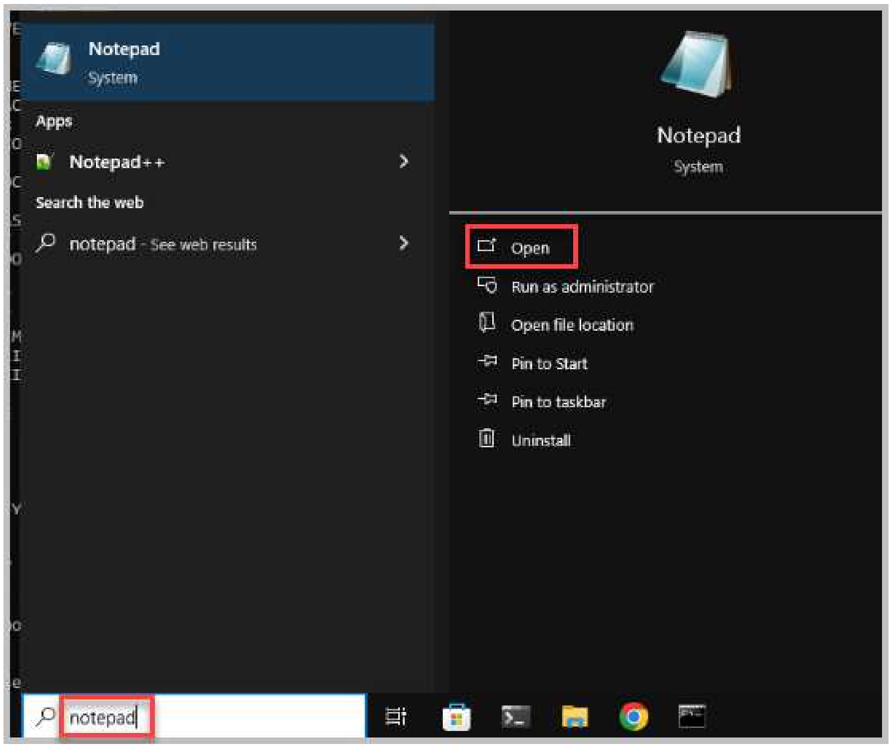
3. At the prompt, type the command below, followed by pressing the **Enter** key. This output is a list of commands available in the Windows command-line interpreter. Review the list of commands identified below to understand the commands you will be exploring in the batch file. The **HELP | MORE** command shows all available commands and what each one does. In the list, review *echo, findstr, rem, systeminfo, and wmic.*

```
HELP | MORE
```



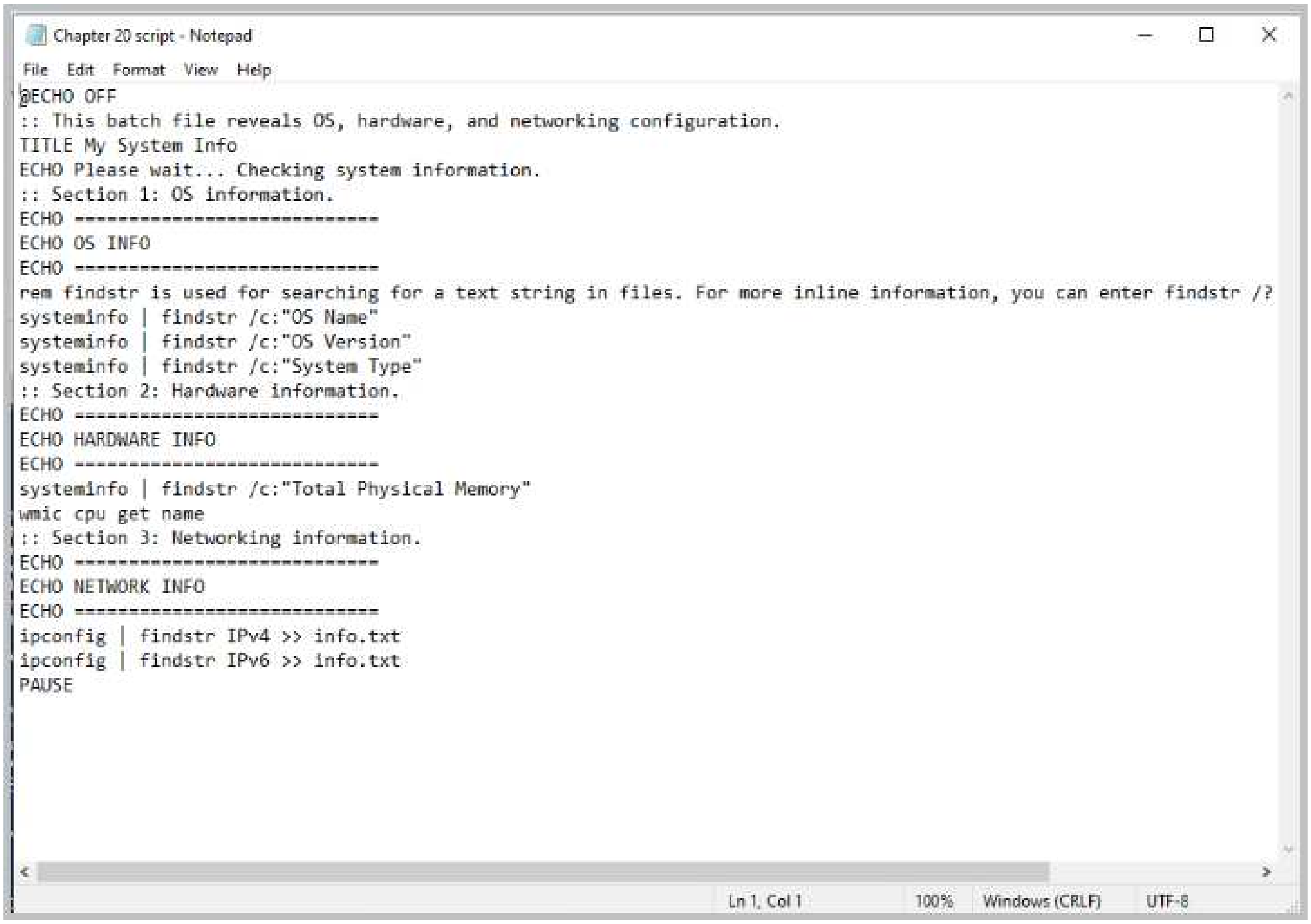
When looking through the list, press the **Spacebar** to move down the list per page or the **Enter** key to go down the list per line. Press **Q** to quit viewing the list at any time.

4. Once you have finished reviewing the command prompt's output, in the **search** textbox, type `notepad` and click **Open.**



5. Enter the following text into the *Notepad* file.

```
@ECHO OFF
:: This batch file reveals OS, hardware, and networking configuration.
TITLE My System Info
ECHO Please wait... Checking system information.
:: Section 1: OS information.
ECHO ============================
ECHO OS INFO
ECHO ============================
rem findstr is used for searching for a text string in files. For more inline
information, you can enter findstr /? at the command prompt.
systeminfo | findstr /c:"OS Name"
systeminfo | findstr /c:"OS Version"
systeminfo | findstr /c:"System Type"
:: Section 2: Hardware information.
ECHO ============================
ECHO HARDWARE INFO
ECHO ============================
systeminfo | findstr /c:"Total Physical Memory"
wmic cpu get name
:: Section 3: Networking information.
ECHO ============================
ECHO NETWORK INFO
ECHO ============================
ipconfig | findstr IPv4 >> info.txt
ipconfig | findstr IPv6 >> info.txt
PAUSE
```


```
@ECHO OFF
:: This batch file reveals OS, hardware, and networking configuration.
TITLE My System Info
ECHO Please wait... Checking system information.
:: Section 1: OS information.
ECHO ============================
ECHO OS INFO
ECHO ============================
rem findstr is used for searching for a text string in files. For more inline information, you can enter findstr /?
systeminfo | findstr /c:"OS Name"
systeminfo | findstr /c:"OS Version"
systeminfo | findstr /c:"System Type"
:: Section 2: Hardware information.
ECHO ============================
ECHO HARDWARE INFO
ECHO ============================
systeminfo | findstr /c:"Total Physical Memory"
wmic cpu get name
:: Section 3: Networking information.
ECHO ============================
ECHO NETWORK INFO
ECHO ============================
ipconfig | findstr IPv4 >> info.txt
ipconfig | findstr IPv6 >> info.txt
PAUSE
```

Breakdown of commands in sysinfo.bat

@ECHO – turns off ECHO

ECHO - print the command currently running on to the command prompt.

Rem- this does not execute it is a remark or note for the programmer

:: - is a comment

Findstr - is used for searching for a text string in files. For more inline information, enter findstr /? at the command prompt

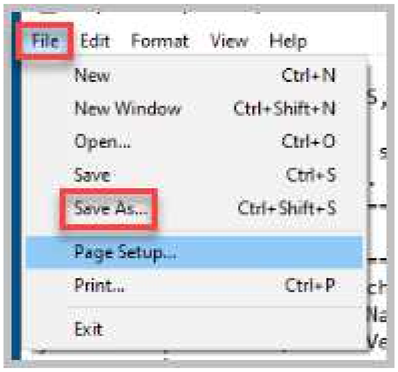
Ipconfig – display network information

>> redirects output to a file if no directory is indicated it is added to the current directory so you will not see the output in the command prompt it will have to be read from the file.

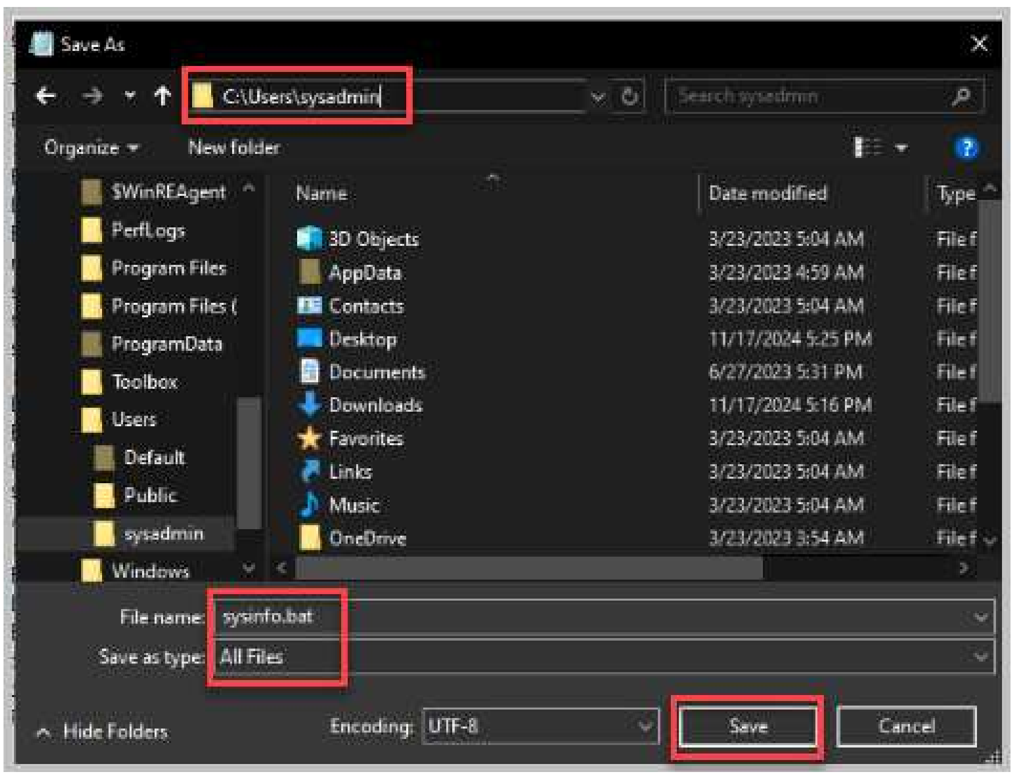
| (key located above the Enter key) - an operator that passes the output of the command before it in the line to the command on the right of it.

PAUSE – requires a key press to continue

6. Click **File > Save As**.

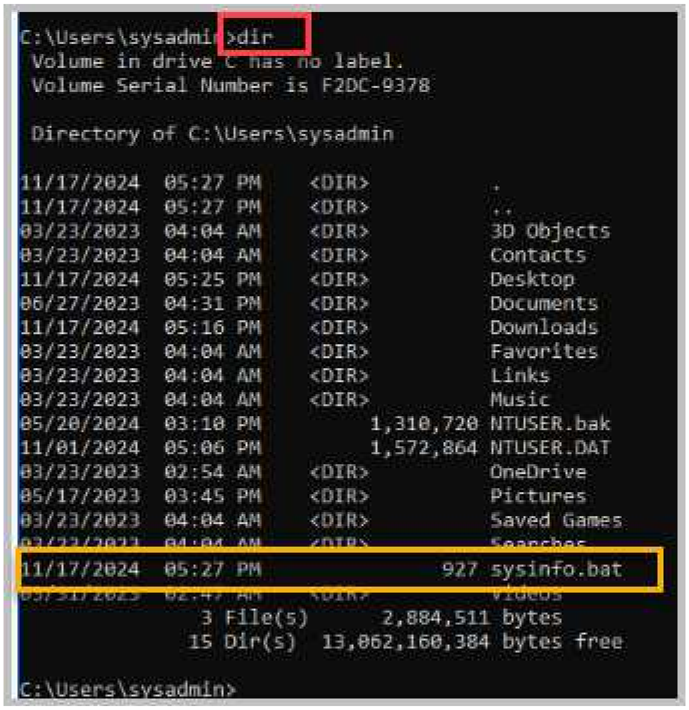


7. Name the file `sysinfo.bat` and save it in the **C:\Users\sysadmin** directory. Verify that the *Save as Type* is listed as **All Files.** Select **Save.**



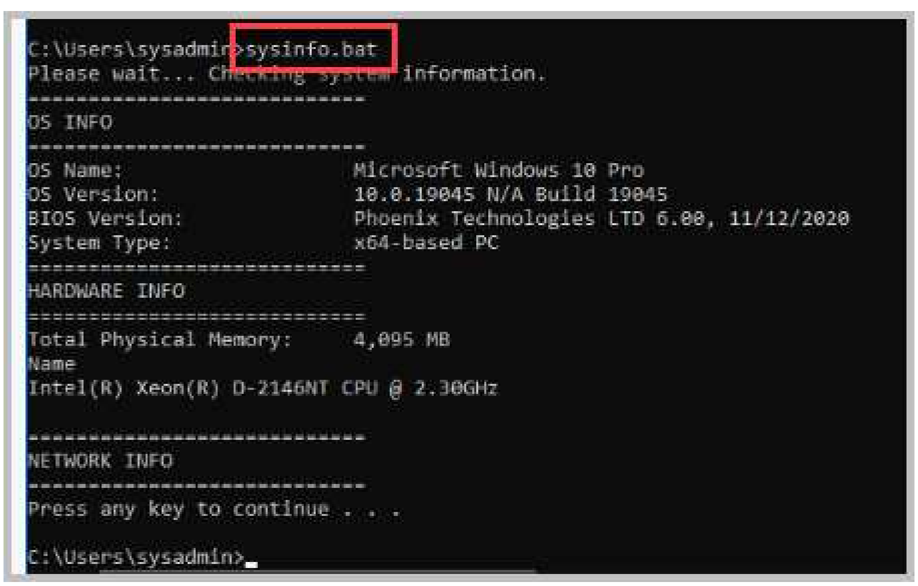
8. Change focus back to the command prompt window and list the contents of the *sysadmin* directory by entering the command below to verify that the file *sysinfo.bat* is saved with the correct file extension. If not, rename the file to sysinfo.bat.

```
dir
```



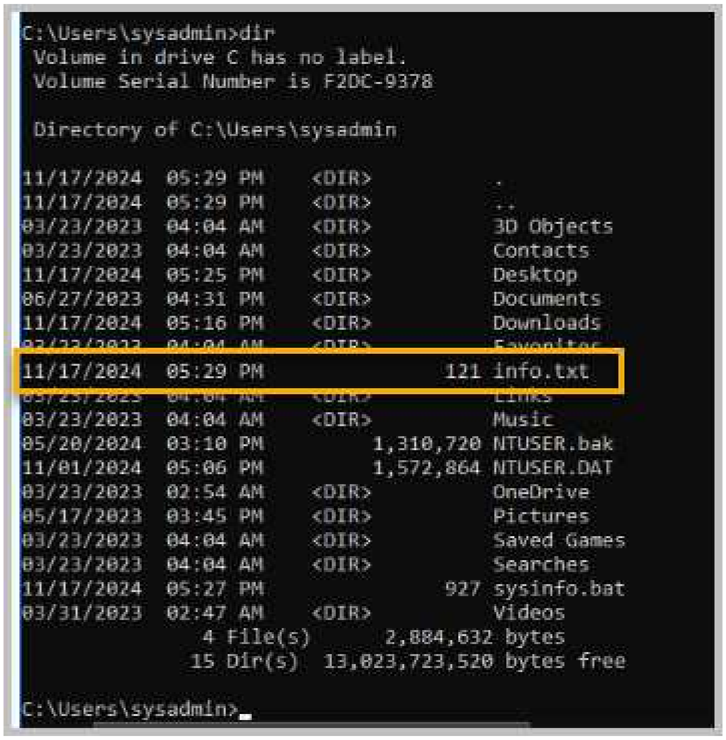
9. At the prompt, enter the command below to run the *sysinfo.bat* script. Carefully watch the output to follow the commands listed in the file. When prompted to press any key to continue, press any key to continue.

```
sysinfo.bat
```



10. List the content of the sysadmin directory to verify that the file *info.txt* has been created as a result of the script. Enter the command below.

```
dir
```



11. At the prompt, type the command **more info.txt** to see the file created by the script. Review the output of the command.

```
more info.txt
```

12. Close the command prompt window.
13. Close the Notepad window.
14. Stay logged in to the *Win10-OS* machine to continue to the next section.

2 Creating and Testing a PowerShell Script

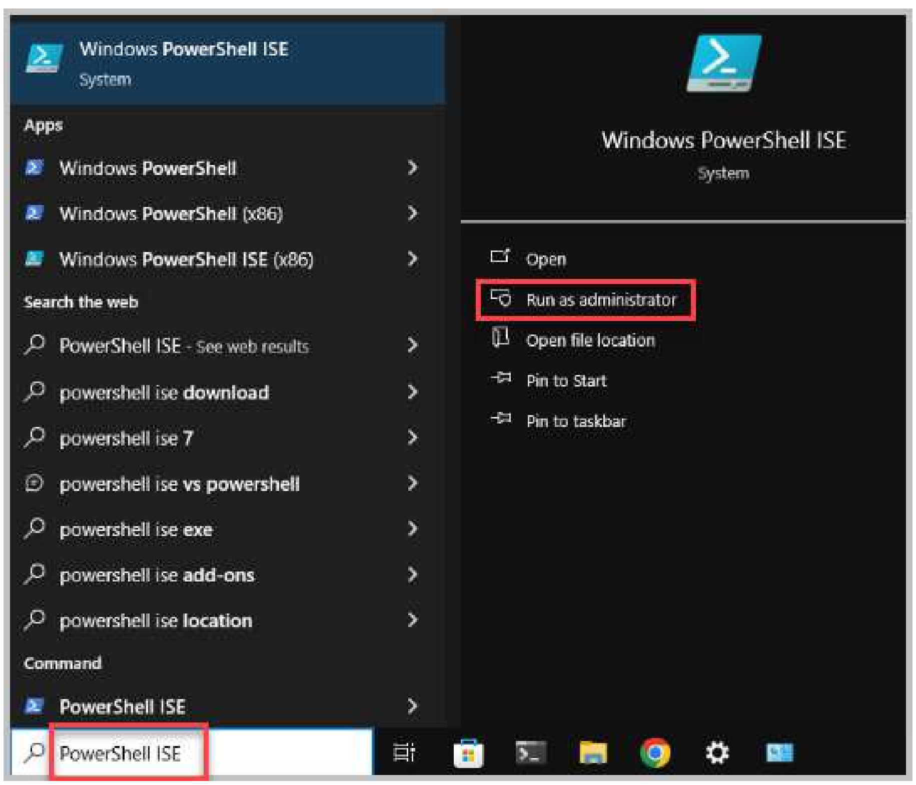
Windows PowerShell is a robust command-line shell and scripting language designed primarily for system administration on Windows. It empowers administrators to efficiently control and automate a wide range of tasks, from simple configuration changes to intricate system management operations.

PowerShell employs a unique approach, utilizing cmdlets (commandlets) instead of traditional commands. Cmdlets are powerful, versatile functions that provide a more structured and efficient way to interact with the system. By chaining cmdlets together in a pipeline, administrators can create complex automation workflows with minimal effort.

Batch files and PowerShell scripts are both collections of commands saved in text files. Batch files utilize the .bat extension and are typically used for simpler, sequential tasks. PowerShell scripts, on the other hand, employ the .ps1 extension and offer significantly greater flexibility and power due to their object-oriented nature and extensive scripting capabilities.

By default, Windows 10 has a security policy that prevents the automatic execution of PowerShell scripts. This safeguard is designed to protect systems from malicious scripts. To run a PowerShell script, administrators must adjust the execution policy to a more permissive setting. This can be done through the PowerShell console or by using the Run as Administrator option.

1. In the *Search Text Box* on the taskbar, type `PowerShell ISE`. Select **Run as administrator**.

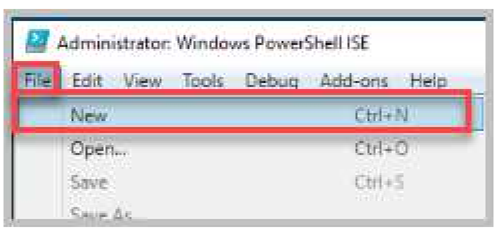


2. Click **Yes** to accept the *User Account Control*.

3. At the *PowerShell ISE* prompt, navigate to the *sysadmin* home directory using the **cd** command. Type `cd c:\users\sysadmin` and select **Enter** on your keyboard.

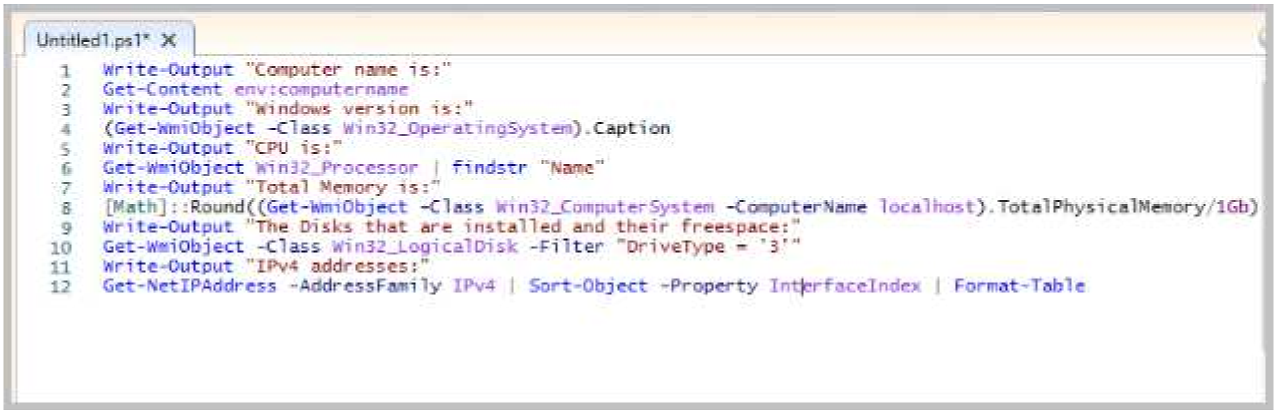
```
cd c:\users\sysadmin
```

4. In the *PowerShell ISE* window, click on **File**, then click **New** to create a new script.



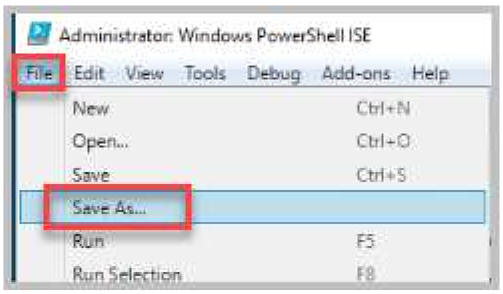
5. Enter the following commands into the newly created *Untitled.ps1* window.

```
Write-Output "Computer name is:"
Get-Content env:computername
Write-Output "Windows version is:"
(Get-WmiObject -Class Win32_OperatingSystem).Caption
Write-Output "CPU is:"
Get-WmiObject Win32_Processor | findstr "Name"
Write-Output "Total Memory is:"
[Math]::Round((Get-WmiObject -Class Win32_ComputerSystem -ComputerName
localhost).TotalPhysicalMemory/1Gb)
Write-Output "The Disks that are installed and their freespace:"
Get-WmiObject -Class Win32_LogicalDisk -Filter "DriveType = '3'"
Write-Output "IPv4 addresses:"
Get-NetIPAddress -AddressFamily IPv4 | Sort-Object -Property InterfaceIndex |
Format-Table
```

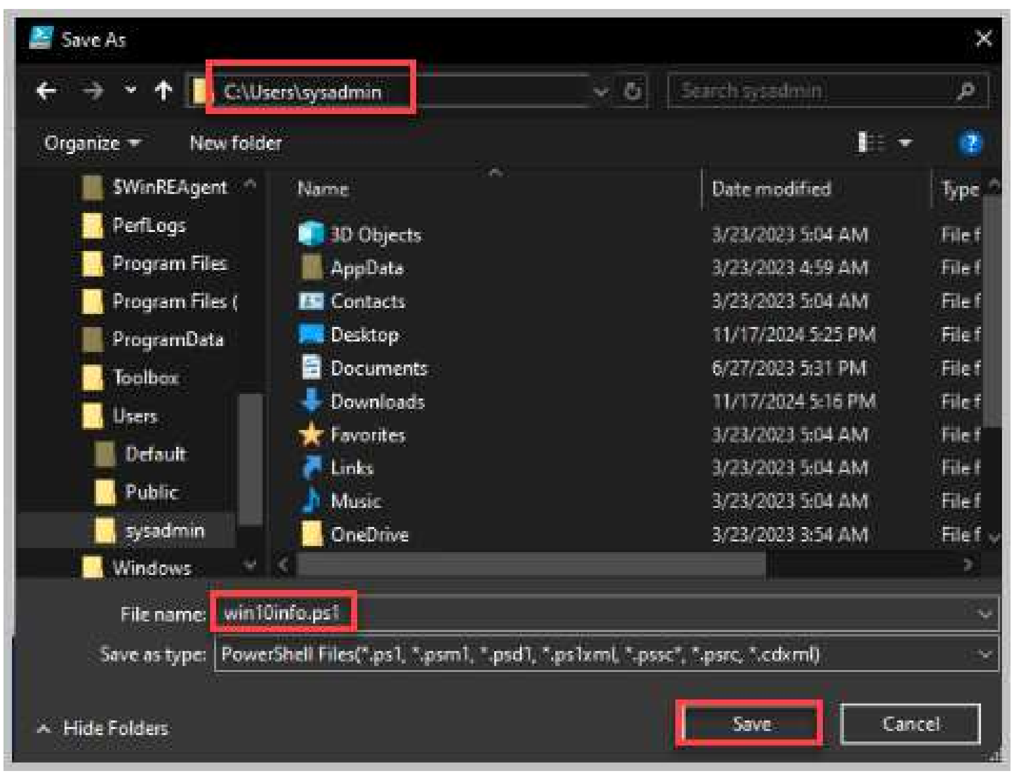



```
Write-Output "Computer name is:"
Get-Content env:computername
Write-Output "Windows version is:"
(Get-WmiObject -Class Win32_OperatingSystem).Caption
Write-Output "CPU is:"
Get-WmiObject Win32_Processor | findstr "Name"
Write-Output "Total Memory is:"
[Math]::Round((Get-WmiObject -Class Win32_ComputerSystem -ComputerName localhost).TotalPhysicalMemory/1Gb)
Write-Output "The Disks that are installed and their freespace:"
Get-WmiObject -Class Win32_LogicalDisk -Filter "DriveType = '3'"
Write-Output "IPv4 addresses:"
Get-NetIPAddress -AddressFamily IPv4 | Sort-Object -Property InterfaceIndex | Format-Table
```

6. Once all the lines are typed, click **File** on the menu bar and select **Save As**.

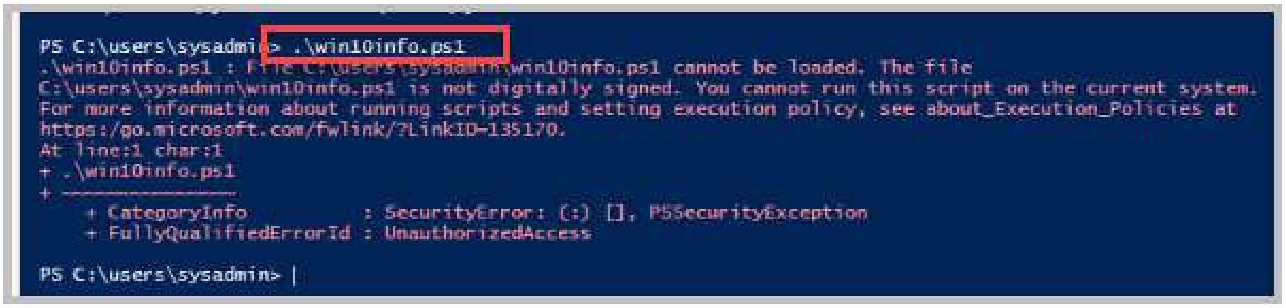


7. In the *Save As* window, name the file `win10info.ps1`, and be sure to save it to the **C:\Users\sysadmin** home directory.



8. Back on the *PowerShell ISE* prompt, type `.\win10info.ps1` command, followed by pressing the **Enter** key to run the script.

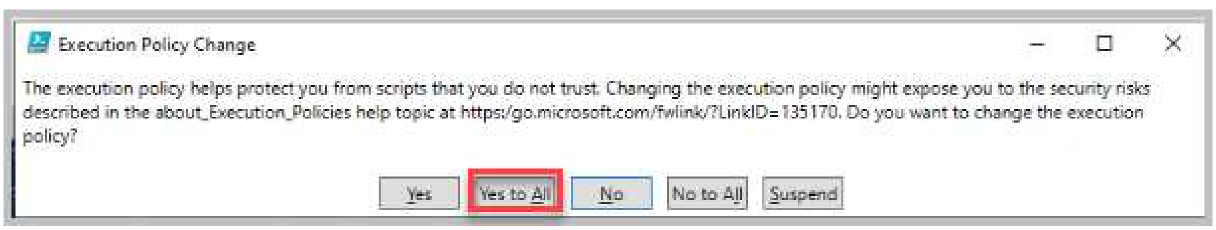
```
.\win10info.ps1
```



9. Notice that an error appears signaling that this script is not digitally signed and that scripts are disabled on the system by default. To disable the security feature, enter the command below.

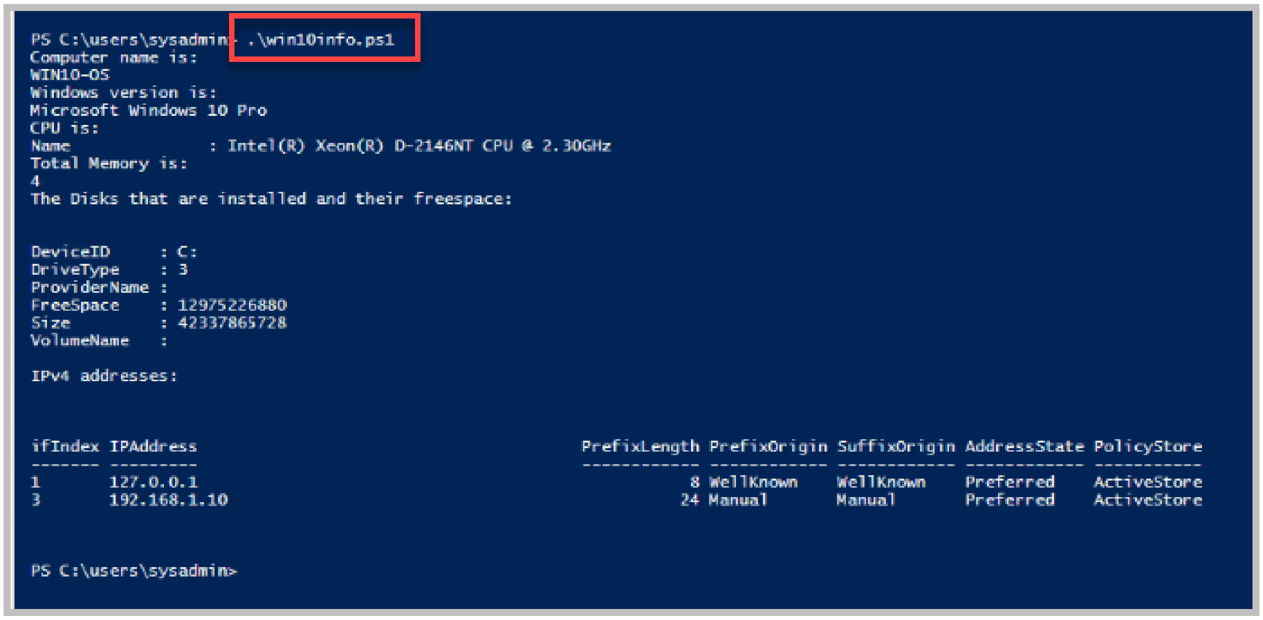
```
Set-ExecutionPolicy RemoteSigned
```

10. A pop-up window appears. Click **Yes to All** to continue.



11. Back on the *PowerShell ISE* prompt, reenter the command below to launch the script.

```
.\win10info.ps1
```



12. Review the output of the script. What comparisons do you see between the Batch file and the PowerShell script? Notice that *ECHO* in the batch file is similar to *Write-Output* in *PowerShell (PS)*. Also, *%computername%* in the batch represents a variable like *Get-Content* reads the *env:computername*. CPU is: *%PROCESSOR_IDENTIFIER%* is like *Get-WmiObject Win32_Processor | findstr "Name"*. Try to find the other batch commands that are equal the PowerShell commands and cmdlets.

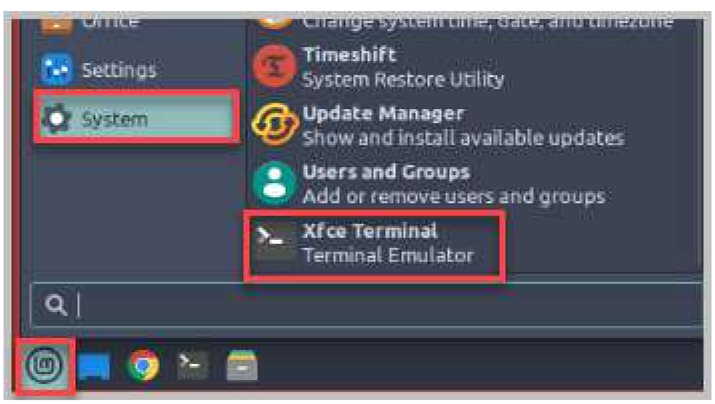
3 Creating a BASH Script using Linux Mint and Testing a Bash Script

BASH is a command interpreter used in Linux and Unix distributions (distros). *Windows Subsystem for Linux* is now also available as a feature in *Windows 10 x64* only. According to Microsoft, "*Windows Subsystem for Linux* enables you to run native *Linux* command-line tools directly on *Windows*, alongside your traditional *Windows* desktop and modern store apps," and it allows you to use the *BASH* shell. *BASH* shell is the environment you type commands that will enable you to interact with your computer. It is a way for you to have the computer do a task, calculation, or something you want it to do. *BASH* shell scripting is a way to use the *BASH* shell interaction to automate command execution with a single command or group of commands.

In this exercise, you will create a *BASH* script and use the *BASH* shell on the *Linux-OS* system to run it. To create a *BASH* script like a batch file in *Windows*, you will need to use a text editor. In *Windows 10,* you use *Notepad*; on the *Linux-OS* system, you will use a built-in text editor tool called *nano*.

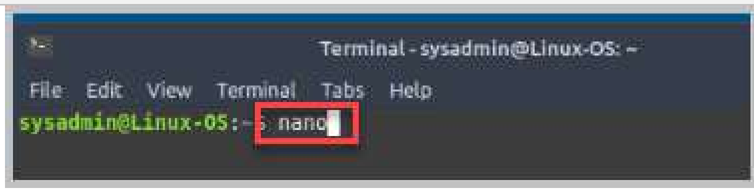
1. Access the **Linux-OS** system from the pod topology.

2. Click on the **Applications** button located on the bottom taskbar and navigate to **System > Xfce Terminal** to open the terminal window, which, in comparison to *Windows*, is similar to the command prompt with regards to using the command-line interface.

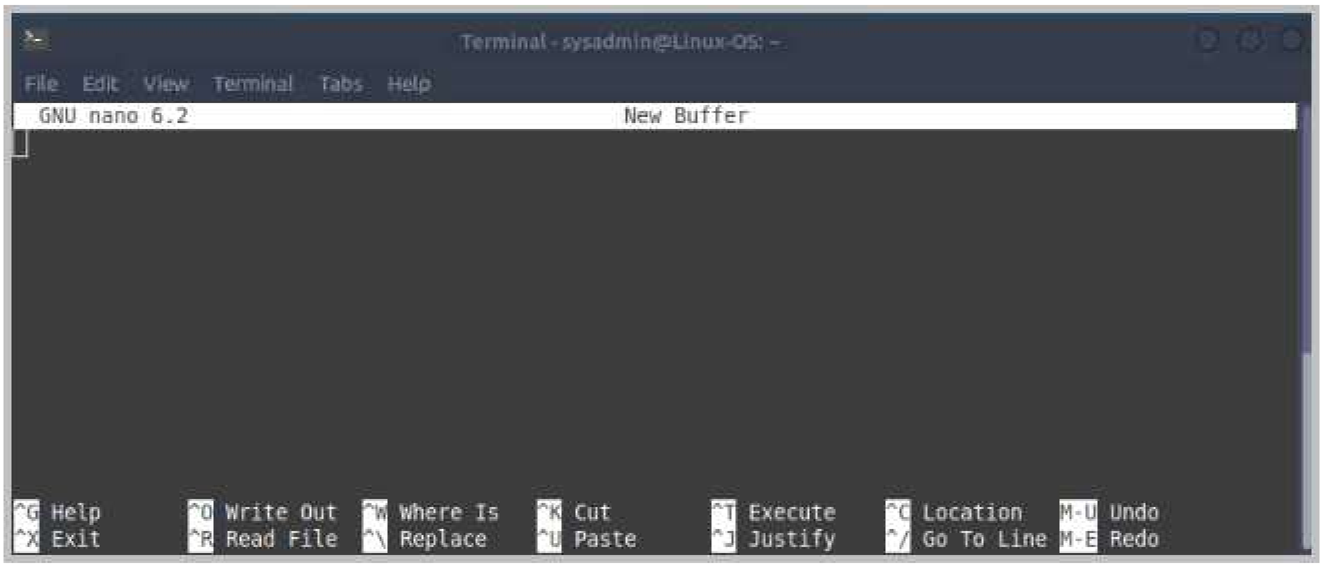


3. At the prompt, type the command below, followed by pressing the **Enter** key to open the nano text editor.

```
nano
```

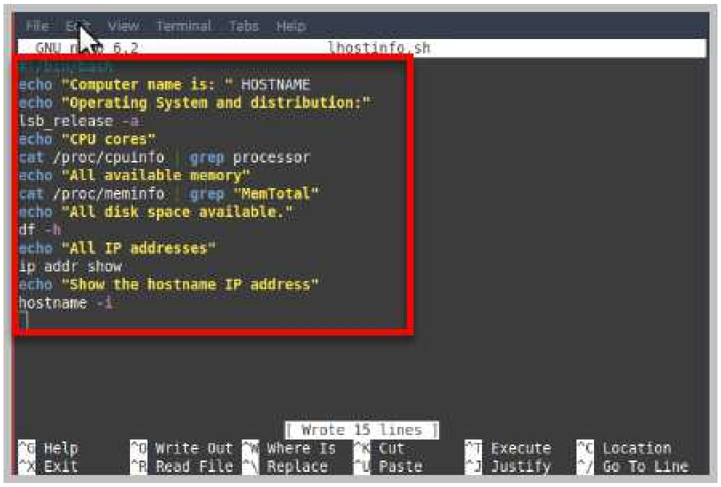


4. GNU nano 6.2 is a user-friendly text editor designed for the command-line environment. It's particularly popular on Unix-like systems, including Linux and macOS.



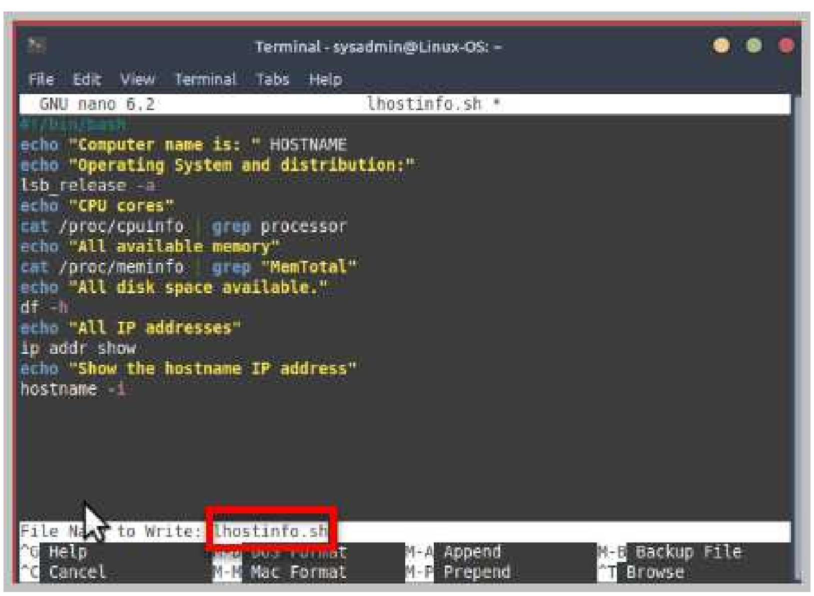
5. You will use the *nano* text editor tool to create a file named *lhostinfo.sh* with the following text. Then you will save it in your home directory. First, start by creating the file by typing the following lines in the *nano* text editor.

```
#!/bin/bash
echo "Computer name is: "  HOSTNAME
echo "Operating System and distribution:"
lsb_release -a
echo "CPU cores"
cat /proc/cpuinfo | grep processor
echo "All available memory"
cat /proc/meminfo | grep  "MemTotal"
echo "All disk space available."
df -h
echo "All IP addresses"
ip addr show
echo “Show the hostname IP address.”
hostname -i
```



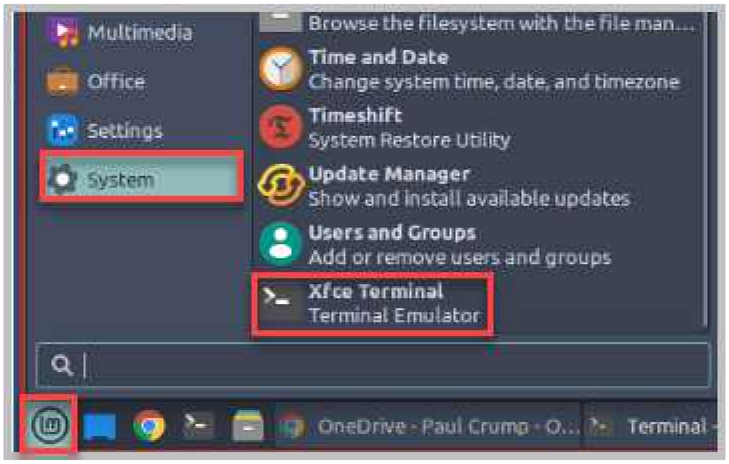
#!/bin/bash - #! Called shebang, it is the syntax used in scripts to indicate an interpreter for execution in this case bash.
lsb – a utility that displays Linux Standard Base information about the Linux distribution.
cat – short for concatenate, is a command which reads data from the file and gives the content as output.
/proc – virtual files that contain dynamic information about the system.
df – short for disk filesystem, this command will get a summary of the available and used disk space on your system.
ip addr – Change to show all IP addresses on the system.

6. To save a file, use the keyboard shortcut **Ctrl+O**. With this key combination, the editor will ask you to provide a filename. Use `lhostinfo.sh` to name your file.



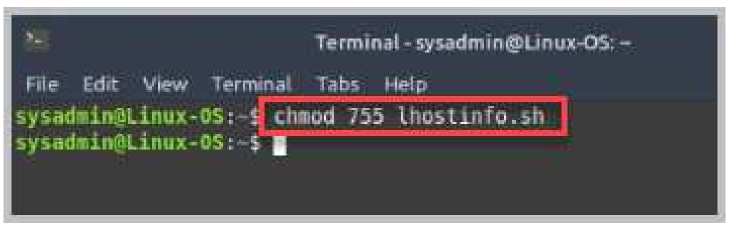
7. Press **Enter** to save the file to the sysadmin's home directory.

8. Open a second terminal by selecting the **Start** button, **System > Xfce Terminal**.



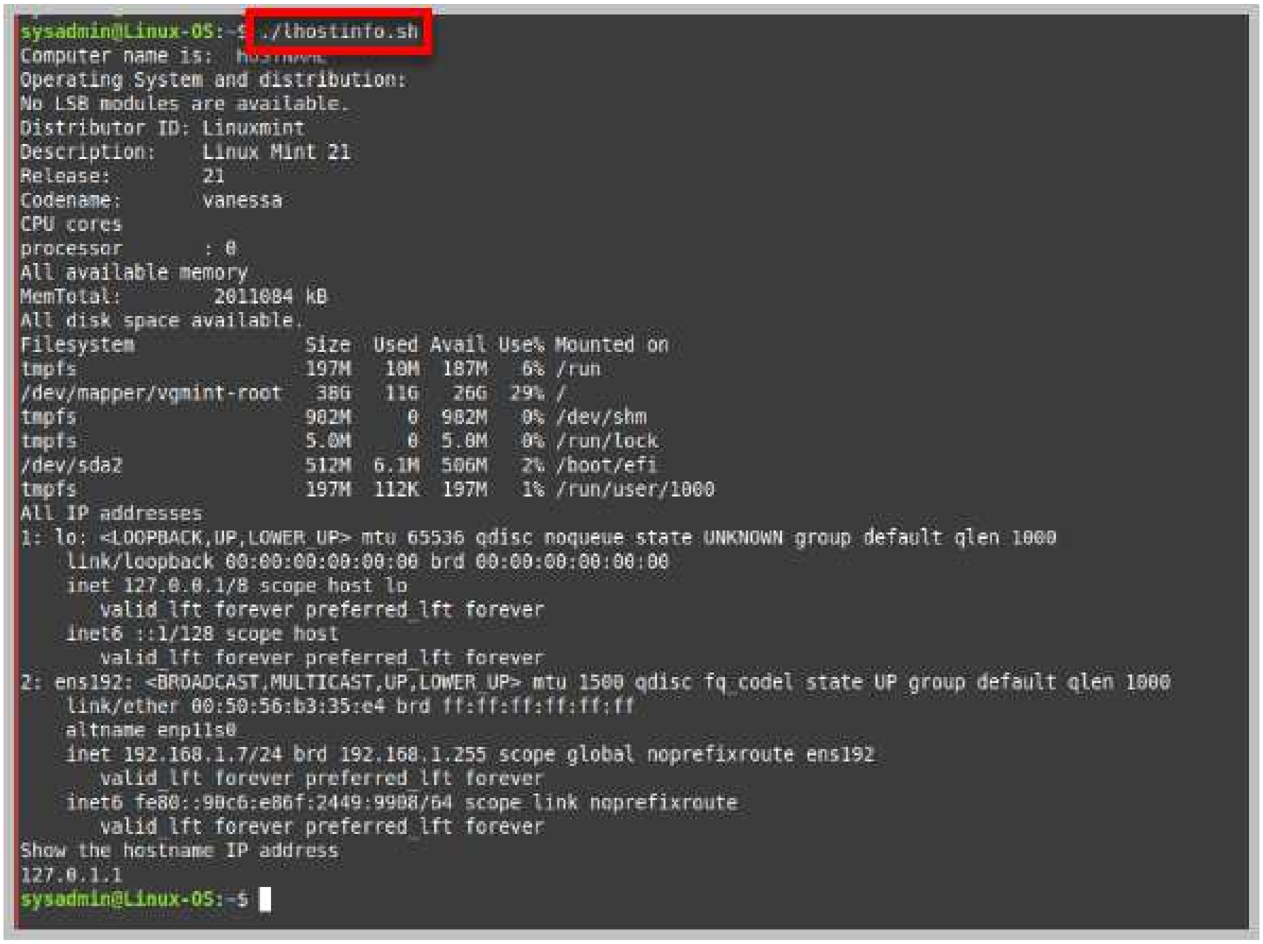
9. You will need to make the script executable. Enter the command below in the newly opened terminal. Press **Enter** to save the file permission changes.

```
chmod 755 lhostinfo.sh
```



10. In the same terminal, enter the command below to execute the script. You will need to use **./** to tell the *BASH* interpreter where the script is located. There is a fixed order in which the BASH interpreter checks directories to find and run a command. If your script is not in one of these directories, then it will not find your script unless you specify the path. The **./** is telling the *BASH* interpreter that the script is in the current directory, but you could also run the script from any other directory by using the full path. Press **Enter** to execute the command.

```
./lhostinfo.sh
```



11. The lab is now complete; you may end the reservation.

